

Concurrency Control in DBMS

Why Concurrency Control Is Needed

Concurrency control is the DBMS mechanism that allows many transactions to execute at the same time while keeping the database correct.

Without concurrency control, interleaved transactions can produce incorrect results even when every transaction is individually correct.

Main goals:

- Maintain database consistency.
- Preserve transaction isolation.
- Improve throughput by allowing parallel execution.
- Avoid incorrect interleavings such as lost update, dirty read, non-repeatable read, and phantom read.
- Ensure schedules behave like some serial order of transactions whenever required.

Transaction Basics

A transaction is a sequence of database operations treated as one logical unit of work.

Common operations:

- `read(X)`: Read data item `X`.
- `write(X)`: Modify data item `X`.
- `commit`: Permanently save transaction changes.
- `rollback` or `abort`: Undo transaction changes.

Example:

```
T1:
read(A)
A = A - 100
write(A)
read(B)
B = B + 100
write(B)
commit
```

This could represent transferring 100 from account `A` to account `B`.

ACID and Concurrency

Concurrency control mainly protects the `I` in ACID, but it also affects the others.

Property	Meaning	Relation to concurrency
Atomicity	All or nothing	Partial updates must not be visible permanently
Consistency	Valid state to valid state	Interleavings must not violate constraints
Isolation	Concurrent transactions should not interfere incorrectly	Main responsibility of concurrency control
Durability	Committed changes survive failures	Handled by recovery/logging, but commit order matters

Schedule

A schedule is the order in which operations of multiple transactions execute.

Serial Schedule

A serial schedule executes one transaction completely before another starts.

```
T1: read(A) write(A) commit
T2: read(A) write(A) commit
```

Serial schedules are simple and correct, but they waste concurrency.

Concurrent Schedule

A concurrent schedule interleaves operations of transactions.

```
T1: read(A)
T2: read(A)
T1: write(A)
T2: write(A)
```

Concurrent schedules improve performance but can cause anomalies.

Serializability

Serializability means a concurrent schedule produces the same result as some serial schedule.

It is the gold standard for correctness in concurrent transaction execution.

Conflict Serializability

Two operations conflict if:

- They belong to different transactions.
- They access the same data item.

- At least one operation is a write.

Conflicting pairs:

- `read(X)` and `write(X)`
- `write(X)` and `read(X)`
- `write(X)` and `write(X)`

Non-conflicting pair:

- `read(X)` and `read(X)`

A schedule is conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting operations.

Precedence Graph

To test conflict serializability:

1. Create one node for each transaction.
2. Add an edge $T_i \rightarrow T_j$ if an operation of T_i conflicts with and occurs before an operation of T_j .
3. If the graph has no cycle, the schedule is conflict serializable.
4. If the graph has a cycle, it is not conflict serializable.

Interview shortcut:

- No cycle means serializable.
- Cycle means not conflict serializable.

Common Concurrency Problems

Lost Update

Lost update occurs when two transactions read the same value and both update it, but one update overwrites the other.

Example:

```
Initial X = 100

T1: read(X)           // X = 100
T2: read(X)           // X = 100
T1: X = X + 10
T1: write(X)          // X = 110
T2: X = X - 20
T2: write(X)          // X = 80
```

Correct result if both updates happened:

```
100 + 10 - 20 = 90
```

Actual result:

```
80
```

The update made by T1 is lost.

Why It Happens

Both transactions read an old value before either transaction finishes. The later write overwrites the earlier write.

How To Prevent Lost Update

- Exclusive locks on data before writing.
- Strict Two-Phase Locking.
- Serializable isolation.
- Repeatable Read in many DBMS implementations.
- Optimistic concurrency control with version checking.
- MVCC with write-write conflict detection.

Interview Line

Lost update is a write-write conflict where one transaction's update is overwritten by another transaction because both worked on the same old value.

Dirty Read

Dirty read occurs when a transaction reads data written by another transaction that has not yet committed.

Example:

```
Initial balance A = 1000

T1: read(A)
T1: A = A - 500
T1: write(A)           // A = 500, not committed

T2: read(A)           // reads 500

T1: rollback          // A restored to 1000
```

T2 read a value that never actually became permanent.

Why It Is Dangerous

If **T2** makes decisions using dirty data, the system may produce incorrect results.

Example:

- Bank transaction temporarily reduces balance.
- Another transaction reads the reduced balance.
- First transaction aborts.
- Second transaction has already acted on false data.

How To Prevent Dirty Read

- Read Committed isolation or stronger.
- Strict 2PL.
- Do not allow reads of uncommitted writes.
- MVCC reads only committed versions.

Interview Line

Dirty read means reading uncommitted data. If the writer rolls back, the reader has seen a value that never existed in the committed database.

Non-Repeatable Read

Non-repeatable read occurs when a transaction reads the same row twice and gets different values because another committed transaction updated or deleted that row in between.

Example:

```
Initial salary of employee 10 = 50000

T1: read salary where emp_id = 10      // 50000

T2: update salary of emp_id = 10 to 60000
T2: commit

T1: read salary where emp_id = 10      // 60000
```

Inside the same transaction, **T1** sees two different values for the same row.

Dirty Read vs Non-Repeatable Read

Dirty read	Non-repeatable read
Reads uncommitted data	Reads committed data
Writer may rollback	Writer has committed

Dirty read	Non-repeatable read
Prevented by Read Committed	Prevented by Repeatable Read or Serializable

How To Prevent Non-Repeatable Read

- Repeatable Read isolation or stronger.
- Shared locks held until transaction end.
- MVCC transaction-level snapshot.

Interview Line

Non-repeatable read means reading the same existing row twice and seeing different committed values because another transaction modified it between reads.

Phantom Read

Phantom read occurs when a transaction repeats a range query and sees a different set of rows because another committed transaction inserted, deleted, or updated rows matching the condition.

Example:

```
T1: select count(*) from Employee where dept = 'HR'
Result: 5

T2: insert into Employee values (101, 'Asha', 'HR')
T2: commit

T1: select count(*) from Employee where dept = 'HR'
Result: 6
```

The new row is a phantom row.

Non-Repeatable Read vs Phantom Read

Non-repeatable read	Phantom read
Same row changes	Set of rows changes
Caused by update/delete of an existing row	Caused by insert/delete/update affecting a search condition
Row-level locks can prevent it	Requires range locks, predicate locks, index locks, or serializable MVCC

How To Prevent Phantom Read

- Serializable isolation.
- Predicate locks.

- Range locks or next-key locks.
- Index-range locking.
- Serializable MVCC with conflict detection.

Interview Line

Phantom read is a range-query anomaly where repeating the same condition returns extra or missing rows because another transaction changed the set of matching rows.

Summary of Read Phenomena

Problem	What changes?	Caused by	Example
Lost update	Final written value	Two writers overwrite each other	Two users update same balance
Dirty read	Uncommitted value is read	Read before writer commits	Read value later rolled back
Non-repeatable read	Same row value changes	Committed update/delete by another transaction	Salary read twice changes
Phantom read	Matching row set changes	Committed insert/delete/update by another transaction	Count query returns more rows

Locks

Locks are concurrency control mechanisms used to control access to data items.

The DBMS grants locks before operations and releases them according to a protocol.

Types of Locks

Binary Lock

A binary lock has only two states:

- Locked
- Unlocked

If an item is locked, no other transaction can access it.

This is simple but too restrictive.

Shared Lock

A shared lock is used for reading.

Notation:

lock-S(X)

Multiple transactions can hold shared locks on the same item at the same time.

Example:

```
T1: lock-S(A)
T2: lock-S(A)
```

Both can read A.

Exclusive Lock

An exclusive lock is used for writing.

Notation:

```
lock-X(X)
```

Only one transaction can hold an exclusive lock on a data item.

If a transaction has an exclusive lock, no other transaction can read or write that item.

Lock Compatibility Matrix

Requested / Held	Shared lock	Exclusive lock
Shared lock	Compatible	Not compatible
Exclusive lock	Not compatible	Not compatible

Meaning:

- Many readers can read the same data item.
- A writer needs exclusive access.
- A reader and writer cannot access the same item at the same time if strict locking is used.

Locking Example

```
T1:
lock-X(A)
read(A)
A = A - 100
write(A)
unlock(A)
commit
```

The exclusive lock ensures nobody else can read or write A while T1 is modifying it.

Lock Conversion

Sometimes a transaction first reads a value and later writes it.

Lock Upgrade

Changing a shared lock to an exclusive lock.

```
lock-S(X)
read(X)
upgrade lock-S(X) to lock-X(X)
write(X)
```

Upgrade can cause deadlock if two transactions both hold shared locks and both wait to upgrade.

Lock Downgrade

Changing an exclusive lock to a shared lock.

```
lock-X(X)
write(X)
downgrade lock-X(X) to lock-S(X)
read(X)
```

Downgrades are less problematic.

Granularity of Locks

Locks can be taken at different levels:

- Database-level lock
- Table-level lock
- Page-level lock
- Row-level lock
- Attribute-level lock
- Predicate or range lock

Coarse-Grained Locking

Locks large objects such as a table or database.

Advantages:

- Simple.
- Low lock-management overhead.

Disadvantages:

- Less concurrency.

Fine-Grained Locking

Locks small objects such as rows.

Advantages:

- More concurrency.

Disadvantages:

- More overhead.
- More complex deadlock handling.

Intention Locks

Intention locks are used in multiple granularity locking. They indicate that a transaction intends to lock lower-level items.

Common intention locks:

- **IS**: Intention Shared
- **IX**: Intention Exclusive
- **SIX**: Shared and Intention Exclusive

Example:

If a transaction wants an exclusive lock on a row, it may first take an intention exclusive lock on the table.

Purpose:

- Prevent conflicts between table-level and row-level locking.
- Help the DBMS quickly detect incompatible locks.

Deadlock

Deadlock occurs when two or more transactions wait forever for locks held by each other.

Example:

```
T1: lock-X(A)
T2: lock-X(B)
T1: requests lock-X(B) and waits
T2: requests lock-X(A) and waits
```

T1 waits for **T2**, and **T2** waits for **T1**.

Deadlock Detection

The DBMS builds a wait-for graph.

- Nodes are transactions.
- Edge $T_i \rightarrow T_j$ means T_i is waiting for T_j .
- A cycle means deadlock.

Deadlock Prevention

Common schemes:

Wait-Die

- Older transaction waits for younger transaction.
- Younger transaction aborts if it requests a lock held by older transaction.

Wound-Wait

- Older transaction aborts younger transaction.
- Younger transaction waits for older transaction.

Deadlock Recovery

The DBMS chooses a victim transaction to abort.

Victim selection factors:

- Amount of work already done.
- Number of locks held.
- Number of transactions affected.
- Transaction priority.

Starvation

Starvation occurs when a transaction waits for a very long time because other transactions keep getting priority.

Prevention:

- Fair lock scheduling.
- Aging.
- First-come-first-served lock queues.

Two-Phase Locking

Two-Phase Locking, or 2PL, is a locking protocol that guarantees conflict serializability.

It has two phases:

1. Growing phase
2. Shrinking phase

Basic 2PL

Growing Phase

The transaction can acquire locks but cannot release any lock.

```
lock-S(A)
lock-X(B)
```

Shrinking Phase

The transaction can release locks but cannot acquire any new lock.

```
unlock(A)
unlock(B)
```

Once a transaction releases its first lock, it enters the shrinking phase. After that, it cannot acquire new locks.

Example of 2PL

```
T1:
lock-X(A)
read(A)
write(A)
lock-X(B)
read(B)
write(B)
unlock(A)
unlock(B)
commit
```

This follows 2PL because all locks are acquired before any lock is released.

Example Violating 2PL

```
T1:
lock-X(A)
read(A)
write(A)
unlock(A)
lock-X(B)      // violation
read(B)
write(B)
unlock(B)
commit
```

This violates 2PL because T1 acquires a new lock after releasing a lock.

Properties of Basic 2PL

Basic 2PL:

- Guarantees conflict serializability.
- Does not guarantee freedom from deadlock.
- Does not guarantee recoverability.
- Can allow cascading rollback.

Important interview point:

2PL guarantees conflict serializability, but it can still lead to deadlocks.

Cascading Rollback

Cascading rollback occurs when one transaction aborts and causes other transactions to abort because they read its uncommitted data.

Example:

```
T1: lock-X(A)
T1: write(A)
T1: unlock(A)

T2: lock-S(A)
T2: read(A)      // reads uncommitted value written by T1
T2: unlock(A)

T1: rollback
T2: must rollback too
```

Basic 2PL can allow this because it may release exclusive locks before commit.

Strict 2PL

Strict Two-Phase Locking is a stronger version of 2PL.

In Strict 2PL:

- A transaction follows 2PL.
- All exclusive locks are held until commit or abort.

Common practical definition:

- All write locks are released only after commit or rollback.

Many DBMS systems use a variant close to strict 2PL.

Example of Strict 2PL

```
T1:  
lock-X(A)  
read(A)  
write(A)  
lock-X(B)  
read(B)  
write(B)  
commit  
unlock(A)  
unlock(B)
```

No other transaction can read uncommitted writes from T1.

Benefits of Strict 2PL

Strict 2PL:

- Guarantees conflict serializability.
- Prevents dirty reads.
- Prevents cascading rollback.
- Produces strict schedules.
- Simplifies recovery.

Limitations:

- Can cause deadlocks.
- May reduce concurrency because locks are held longer.

Rigorous 2PL

Rigorous 2PL is even stricter.

In Rigorous 2PL:

- All locks, both shared and exclusive, are held until commit or abort.

Comparison:

Protocol	Shared locks released	Exclusive locks released
Basic 2PL	During shrinking phase	During shrinking phase
Strict 2PL	May be released earlier	At commit/abort
Rigorous 2PL	At commit/abort	At commit/abort

Conservative 2PL

Conservative 2PL, also called static 2PL, requires a transaction to acquire all required locks before it begins execution.

Advantage:

- Prevents deadlock.

Disadvantage:

- Requires knowing all needed locks in advance.
- Reduces concurrency.

2PL vs Strict 2PL

Feature	Basic 2PL	Strict 2PL
Guarantees conflict serializability	Yes	Yes
Prevents dirty reads	Not always	Yes
Prevents cascading rollback	No	Yes
Holds exclusive locks until commit	Not required	Required
Deadlock possible	Yes	Yes
Used in real systems	Less common alone	Very common idea

MVCC

MVCC stands for Multi-Version Concurrency Control.

Instead of overwriting a data item immediately, the DBMS keeps multiple versions of data. Readers can read an older committed version while writers create a newer version.

This improves concurrency because readers and writers do not always block each other.

Basic Idea of MVCC

Each row may have multiple versions:

```
Row X:
Version 1: value = 100, created by T1, committed
Version 2: value = 120, created by T2, uncommitted
```

A transaction reads the version that is visible according to its snapshot or timestamp.

MVCC Metadata

A version may store metadata such as:

- Creation transaction ID.
- Deletion transaction ID.
- Commit timestamp.
- Rollback status.

- [Pointer to previous version.](#)

Different DBMS products implement this differently.

MVCC Example

```
Initial X = 100

T1 starts
T1 reads X = 100

T2 starts
T2 updates X to 200
T2 commits

T1 reads X again
```

Under snapshot isolation, **T1** still sees:

```
X = 100
```

because **T1** reads from the snapshot taken when it started.

Under read committed MVCC, **T1** may see:

```
X = 200
```

on the second read because each statement may get a fresh committed snapshot.

MVCC Advantages

- Readers do not block writers in many cases.
- Writers do not block readers in many cases.
- Better performance for read-heavy workloads.
- Natural support for consistent snapshots.
- Helps prevent dirty reads.
- Useful for long-running read queries.

MVCC Disadvantages

- More storage needed for old versions.
- Old versions must be garbage-collected or vacuumed.
- Write-write conflicts still need handling.
- Snapshot isolation is not always fully serializable.
- Implementation is complex.

MVCC and Write Conflicts

MVCC allows multiple readers, but two writers updating the same row still conflict.

Example:

```
T1 reads X = 100  
T2 reads X = 100  
T1 writes X = 110  
T2 writes X = 120
```

The DBMS must decide whether to:

- Block one writer.
- Abort one writer.
- Detect conflict at commit time.
- Use locks for writes.

MVCC does not mean no locking at all. Most MVCC systems still use locks for writes, schema changes, or constraints.

Snapshot Isolation

Snapshot isolation is an isolation approach often implemented using MVCC.

A transaction reads from a consistent snapshot of the database.

Rules:

- Reads see committed data as of the transaction start time.
- Writes may be checked for conflicts at commit time.
- Dirty reads are prevented.
- Non-repeatable reads are usually prevented because the same snapshot is used.
- Some phantom-like effects may be prevented for reads, but write skew can still occur.

Write Skew

Write skew is an anomaly possible under snapshot isolation.

Example:

Rule:

```
At least one doctor must be on call.
```

Initial:

```
Doctor A on_call = true  
Doctor B on_call = true
```

Transactions:

```
T1 reads A and B, sees both on call  
T2 reads A and B, sees both on call  
  
T1 sets A off call  
T2 sets B off call  
  
Both commit
```

Final:

```
A = off call  
B = off call
```

The constraint is violated even though each transaction saw a valid snapshot.

Serializable isolation is needed to prevent this reliably.

Isolation Levels

Isolation levels define how much one transaction is isolated from others.

Standard SQL isolation levels:

1. Read Uncommitted
2. Read Committed
3. Repeatable Read
4. Serializable

Higher isolation gives more correctness but may reduce concurrency.

Read Uncommitted

Read Uncommitted is the weakest isolation level.

Allowed:

- Dirty reads.
- Non-repeatable reads.
- Phantom reads.
- Lost updates may occur depending on implementation.

Example:

```
T1 writes A = 500 but does not commit  
T2 reads A = 500  
T1 rolls back
```

T2 read dirty data.

Use case:

- Rarely recommended.
- Sometimes used for approximate reporting where correctness is less important.

Interview point:

Read Uncommitted provides maximum concurrency but minimum correctness.

Read Committed

Read Committed allows a transaction to read only committed data.

Prevents:

- Dirty reads.

Allows:

- Non-repeatable reads.
- Phantom reads.

Example:

```
T1 reads balance = 1000  
T2 updates balance = 1200  
T2 commits  
T1 reads balance again = 1200
```

This is allowed because 1200 is committed.

Common in real systems:

- Many DBMS systems use Read Committed as the default or common isolation level.

Repeatable Read

Repeatable Read ensures that if a transaction reads the same row again, it sees the same value.

Prevents:

- Dirty reads.

- Non-repeatable reads.

May allow:

- Phantom reads in the SQL standard definition.

Important note:

Actual behavior differs by DBMS. For example, some systems using MVCC or next-key locks prevent many phantom reads under Repeatable Read.

Serializable

Serializable is the strongest standard isolation level.

It guarantees that concurrent transactions behave as if they ran one after another in some serial order.

Prevents:

- Dirty reads.
- Non-repeatable reads.
- Phantom reads.
- Many other serialization anomalies.

Implementation methods:

- Strict 2PL with predicate or range locks.
- Serializable MVCC.
- Optimistic concurrency control with validation.

Disadvantages:

- Lower concurrency.
- More blocking or aborts.
- Possible deadlocks with locking-based implementation.

Isolation Level Anomaly Table

SQL standard view:

Isolation level	Dirty read	Non-repeatable read	Phantom read
Read Uncommitted	Possible	Possible	Possible
Read Committed	Prevented	Possible	Possible
Repeatable Read	Prevented	Prevented	Possible
Serializable	Prevented	Prevented	Prevented

Placement memory trick:

```

RU  -> Dirty allowed
RC  -> Dirty prevented
RR  -> Dirty + Non-repeatable prevented
SER -> Dirty + Non-repeatable + Phantom prevented

```

Lost Update and Isolation Levels

Lost update is not always shown in the classic SQL anomaly table, but it is very important in interviews.

Typical prevention:

Isolation level	Lost update behavior
Read Uncommitted	Usually possible
Read Committed	May be possible unless write locks/version checks are used
Repeatable Read	Usually prevented in many DBMS systems
Serializable	Prevented

Important:

Exact behavior depends on the DBMS implementation.

Locks vs MVCC

Topic	Lock-based concurrency	MVCC
Main idea	Control access using locks	Keep multiple versions
Reads	May block writes or be blocked	Often do not block writes
Writes	Require exclusive locks	Still need conflict handling
Dirty reads	Prevented if reads wait for committed data	Prevented by reading committed versions
Storage overhead	Lower	Higher due to versions
Deadlock	Possible	Still possible for writes/locks
Best for	Strong control, serializability	High read concurrency

Recoverable, Cascadeless, and Strict Schedules

These are often asked with concurrency control.

Recoverable Schedule

A schedule is recoverable if a transaction commits only after all transactions whose data it read have committed.

Example:

```
T1 writes X
T2 reads X from T1
T1 commits
T2 commits
```

This is recoverable.

Bad case:

```
T1 writes X
T2 reads X from T1
T2 commits
T1 aborts
```

This is not recoverable because T2 committed based on data that was rolled back.

Cascadeless Schedule

A schedule is cascadeless if transactions only read committed data.

This prevents cascading rollback.

Strict Schedule

A schedule is strict if no transaction can read or write a data item until the last transaction that wrote it has committed or aborted.

Strict schedules are easiest for recovery.

Relationship:

```
Strict schedule => Cascadeless schedule => Recoverable schedule
```

Locking Protocol Comparison

Protocol	Guarantees serializability	Avoids cascading rollback	Avoids deadlock
Basic 2PL	Yes	No	No
Strict 2PL	Yes	Yes	No
Rigorous 2PL	Yes	Yes	No

Protocol	Guarantees serializability	Avoids cascading rollback	Avoids deadlock
Conservative 2PL	Yes	Usually yes	Yes

Common Interview Questions

What is concurrency control?

Concurrency control is the DBMS mechanism that coordinates simultaneous transactions so that database consistency and isolation are preserved.

Why not execute all transactions serially?

Serial execution is correct but inefficient. It wastes CPU, disk, and user waiting time. Concurrency improves throughput and resource utilization.

What is the difference between serial and serializable?

Serial means transactions actually run one after another.

Serializable means transactions may run concurrently, but the final effect is equivalent to some serial order.

Does 2PL guarantee serializability?

Yes. Two-Phase Locking guarantees conflict serializability.

Does 2PL prevent deadlock?

No. Basic 2PL and Strict 2PL can cause deadlocks. Conservative 2PL prevents deadlocks by acquiring all locks before execution.

Why is Strict 2PL better than Basic 2PL?

Strict 2PL holds exclusive locks until commit or abort, so other transactions cannot read uncommitted writes. This prevents dirty reads and cascading rollback.

What is the difference between 2PL and timestamp ordering?

2PL uses locks and may make transactions wait.

Timestamp ordering assigns timestamps and orders conflicting operations based on timestamp order. It may abort transactions that violate the order.

What is the difference between locking and MVCC?

Locking blocks conflicting operations. MVCC keeps multiple versions so readers can often read a consistent older version without blocking writers.

Does MVCC eliminate locks?

No. MVCC reduces read-write blocking, but DBMS systems still use locks for writes, indexes, constraints, schema changes, and conflict detection.

What is phantom read?

A phantom read happens when repeating a range query returns a different set of rows due to another committed transaction inserting, deleting, or updating matching rows.

Why are row locks not enough to prevent phantom reads?

Because phantom reads involve rows that did not exist or did not match the condition during the first query. The DBMS must lock the range or predicate, not just existing rows.

Which isolation level prevents dirty reads?

Read Committed and stronger levels prevent dirty reads.

Which isolation level prevents phantom reads?

Serializable prevents phantom reads according to the SQL standard.

Example Schedules for Practice

Lost Update Schedule

```
T1: read(X)
T2: read(X)
T1: write(X)
T2: write(X)
T1: commit
T2: commit
```

Problem:

T2 overwrites the update made by **T1**.

Dirty Read Schedule

```
T1: write(X)
T2: read(X)
T1: rollback
T2: commit
```

Problem:

T2 read data written by **T1**, but **T1** rolled back.

Non-Repeatable Read Schedule

```
T1: read(X)
T2: write(X)
T2: commit
T1: read(X)
```

Problem:

T1 reads different committed values of X.

Phantom Read Schedule

```
T1: select rows where marks > 90
T2: insert row with marks = 95
T2: commit
T1: select rows where marks > 90
```

Problem:

T1 sees a new row in the second query.

Quick Revision Sheet

Term	One-line meaning
Concurrency control	Keeps concurrent transactions correct
Schedule	Order of operations of transactions
Serial schedule	Transactions execute one by one
Serializable schedule	Concurrent schedule equivalent to serial
Conflict	Same item, different transactions, at least one write
Lost update	One update overwrites another
Dirty read	Reading uncommitted data
Non-repeatable read	Same row read twice gives different values
Phantom read	Same range query gives different row set
Shared lock	Read lock
Exclusive lock	Write lock
2PL	Locks acquired first, released later
Strict 2PL	Holds write locks until commit/abort
MVCC	Multiple versions for concurrent reads/writes

Term	One-line meaning
Read Committed	Prevents dirty reads
Repeatable Read	Prevents dirty and non-repeatable reads
Serializable	Strongest isolation, equivalent to serial execution

Mnemonics

Isolation Levels

```

Read Uncommitted -> can read anything, even dirty data
Read Committed   -> only committed data
Repeatable Read   -> same row stays same
Serializable      -> behaves like serial execution

```

Anomalies

```

Dirty read          -> uncommitted value
Non-repeatable read -> same row, changed value
Phantom read        -> same condition, changed row set
Lost update         -> overwritten write

```

Lock Types

```

Shared lock    -> read together
Exclusive lock -> write alone

```

Final Placement Tips

- Always distinguish dirty read from non-repeatable read using committed vs uncommitted data.
- Always distinguish non-repeatable read from phantom read using same row vs row set.
- Remember that 2PL guarantees conflict serializability but can still deadlock.
- Remember that Strict 2PL prevents cascading rollback by holding write locks until commit or abort.
- Remember that MVCC improves read concurrency but does not remove the need for write conflict handling.
- In anomaly questions, write the schedule step by step. Most answers become obvious once the operations are ordered.
- In isolation-level questions, answer according to the SQL standard first, then mention that real DBMS implementations may differ.